

MongoDB Food Delivery Practical

Restaurants, Orders, Aggregation & Delivery Status

1. Create restaurants Collection and Insert 5 Documents

MongoDB Command:

```
use foodDeliveryDB

db.restaurants.insertMany([
  { _id: "R001", name: "Spice Garden", cuisine: "Indian", location: "Mumbai", average_rating: 4.5 },
  { _id: "R002", name: "Pizza Palace", cuisine: "Italian", location: "Pune", average_rating: 4.2 },
  { _id: "R003", name: "Dragon Wok", cuisine: "Chinese", location: "Delhi", average_rating: 4.6 },
  { _id: "R004", name: "Burger Hub", cuisine: "American", location: "Bangalore", average_rating: 4.1 },
  { _id: "R005", name: "South Delight", cuisine: "South Indian", location: "Chennai", average_rating: 4.7 }
])
```

Expected / Sample Output:

```
Output:
{
  acknowledged: true,
  insertedIds: {
    '0': 'R001', '1': 'R002', '2': 'R003',
    '3': 'R004', '4': 'R005'
  }
}
```

2. Create orders Collection and Insert 3 Sample Orders

MongoDB Command:

```
db.orders.insertMany([
  {
    _id: "O001", user_id: "U001", restaurant_id: "R001",
    items: [
      { item_name: "Butter Chicken", price: 280 },
      { item_name: "Naan", price: 60 }
    ],
    total_amount: 340,
    order_date: ISODate("2026-09-08T13:30:00Z")
  },
  {
    _id: "O002", user_id: "U002", restaurant_id: "R002",
    items: [
      { item_name: "Margherita Pizza", price: 350 },
      { item_name: "Garlic Bread", price: 150 }
    ],
    total_amount: 500,
    order_date: ISODate("2026-09-09T18:45:00Z")
  },
  {
    _id: "O003", user_id: "U003", restaurant_id: "R001",
    items: [
      { item_name: "Biryani", price: 300 },
      { item_name: "Lassi", price: 80 }
    ],
    total_amount: 380,
    order_date: ISODate("2026-09-10T20:15:00Z")
  }
])
```

Expected / Sample Output:

```
Output:
{
  acknowledged: true,
  insertedIds: {
    '0': 'O001', '1': 'O002', '2': 'O003'
  }
}
```

```
}  
}
```

3. Count Orders for Each Restaurant

MongoDB Command:

```
db.orders.aggregate([  
  {  
    $group: {  
      _id: "$restaurant_id",  
      order_count: { $sum: 1 }  
    }  
  }  
])
```

Expected / Sample Output:

```
Output:  
[  
  { _id: "R001", order_count: 2 },  
  { _id: "R002", order_count: 1 }  
]
```

4. Top 2 Restaurants by Average Order Amount

MongoDB Command:

```
db.orders.aggregate([  
  {  
    $group: {  
      _id: "$restaurant_id",  
      average_order_amount: { $avg: "$total_amount" }  
    }  
  },  
  {  
    $sort: { average_order_amount: -1 }  
  },  
  {  
    $limit: 2  
  }  
])
```

Expected / Sample Output:

```
Output:  
[  
  { _id: "R002", average_order_amount: 500 },  
  { _id: "R001", average_order_amount: 360 }  
]
```

Calculation:
 $R001 = (340 + 380) / 2 = 360$
 $R002 = 500 / 1 = 500$

5. Add Random Delivery Status and Count Pending Orders

MongoDB Command:

```
db.orders.updateMany(  
  {},  
  [  
    {  
      $set: {  
        delivery_status: {  
          $arrayElemAt: [  
            ["pending", "out for delivery", "delivered"],  
            {  
              $floor: {  
                $multiply: [{ $rand: {} }, 3]  
              }  
            }  
          ]  
        }  
      }  
    ]  
  )
```

```
)
  ]
}
}
}
]
```

Expected / Sample Output:

Output:

```
{
  acknowledged: true,
  matchedCount: 3,
  modifiedCount: 3
}
```

Example updated orders:

0001 -> delivered

0002 -> pending

0003 -> out for delivery

Count pending orders:

```
db.orders.countDocuments({delivery_status:"pending"})
```

Sample Output:

1

Note: The delivery status is randomly assigned, so the pending count may differ when the command is executed again.

Final Result Summary

Task	Result
Restaurants inserted	5 documents
Orders inserted	3 documents
R001 order count	2
R002 order count	1
Top restaurant by average order	R002 – ■500
Second restaurant by average order	R001 – ■360
Sample pending orders	1 (random assignment)

This practical demonstrates MongoDB document insertion, nested arrays, \$group, \$sum, \$avg, \$sort, \$limit, updateMany with an aggregation pipeline, \$rand, and countDocuments.